

CS-306-MJ-P Practical Exam – Answers & Viva Guide

Oct 9, 2026 · @Faisal

How to use this guide

Each section covers one exam slip (page number from your PDF): Q1 Java code, Q2 web code, then likely viva questions with short answers. This is the master version: for every question it uses the shorter, easier code from either your teacher's notes or the original answers, with bugs in both fixed.

Running Java: save as `ClassName.java`, then `javac ClassName.java` and `java ClassName`. For packages: `javac -d . File.java` then `java pkg.MainClass`.

Running web/JS: HTML files open in a browser. Plain JS files run with `node file.js`. ES modules need `.mjs` or `"type": "module"` in `package.json`.

Viva basics asked on every slip:

- **JDK vs JRE vs JVM:** JDK = tools to develop (javac); JRE = libraries + JVM to run; JVM = executes bytecode.
- **Why is Java platform independent?** Code compiles to bytecode (.class), which any JVM can run.
- **OOP pillars:** Encapsulation, Inheritance, Polymorphism, Abstraction.
- **Why is main() static?** JVM calls it without creating an object.
- **HTML5 vs HTML:** HTML5 adds semantic tags (header, nav, footer), new input types, audio/video, canvas.
- **let vs var vs const:** var is function-scoped; let and const are block-scoped; const cannot be reassigned.
- **Node.js:** JavaScript runtime built on Chrome's V8 engine; single-threaded, non-blocking, event-driven.

Page 1 – Array Sum & Sort | College Registration Form

Q1. Sum of array elements and ascending order

```
import java.util.*;
class Slip1_1
{
    public static void main(String[] args)
    {
        int a[] = {45, 12, 89, 7, 23, 64};
        int sum = 0;
        for (int i = 0; i < a.length; i++)
        {
            sum += a[i];
        }
        System.out.println("Sum = " + sum);
        Arrays.sort(a);
        System.out.println("Ascending: " + Arrays.toString(a));
    }
}
```

Shortcut: you can replace the bubble sort loop with `Arrays.sort(a)`, as done above. If the examiner asks you to sort manually, use bubble sort: two nested loops that swap `a[j]` and `a[j+1]` when `a[j] > a[j+1]`.

Q2. College/School Registration Form (HTML5 + CSS3)

```
<!DOCTYPE html>
<html>
<head>
<title>Registration Form</title>
<style>
    form
    {
        width: 600px;
        margin: auto;
        background: #ffffdd;
        padding: 20px;
    }
    .c
    {
        text-align: center;
```

```

    }
    span
    {
        color: red;
        font-size: 12px;
    }
</style>
</head>
<body>
<form>
    <h1 class="c">My-Future College / School</h1>
    <p class="c">Please Fill Below Detail In Student Registration Form</p>
    <b>Student name :</b> <input type="text" minlength="3" maxlength="10"
required>
    <span>Min 3 & Max 10 letter is allow</span><br><br>
    <b>Date of birth :</b> <input type="date" required>
    <span>Valid Date is allow</span><br><br>
    <b>Mobile no. :</b> <input type="tel" pattern="[0-9]{10}" required>
    <span>Only integer is allow</span><br><br>
    <b>Email id :</b> <input type="email" required>
    <span>Valid Email Id is allow</span><br><br>
    <b>Password :</b> <input type="password" minlength="8" required>
    <span>Min 8 digit is allow</span><br><br>
    <b>Gender :</b>
    <input type="radio" name="g" required> Male
    <input type="radio" name="g"> Female<br><br>
    <b>Hobbies :</b>
    <input type="checkbox"> Drawing <input type="checkbox"> Singing
    <input type="checkbox"> Dancing <input type="checkbox"> Sketch
    <input type="checkbox"> Sports<br><br>
    <b>Course :</b>
    <select required>
        <option value="">--- Select Admission Course's ---</option>
        <option>B.Sc (CS)</option>
        <option>BCA</option>
    </select><br><br>
    <b>Student photo :</b> <input type="file" required><br><br>
    <b>Register Time :</b> <input type="time" required><br><br>
    <b>Address :</b><br>
    <textarea rows="3" cols="30" required></textarea><br><br>
    <input type="checkbox" required> <b>I hereby declare that above
information provided is correct</b>
    <p class="c">

```

```

        <input type="submit" value="Register">
        <input type="reset" value="Reset">
    </p>
</form>
</body>
</html>

```

Viva questions – Page 1

- **What is an array?** A collection of same-type elements stored in contiguous memory, accessed by index starting at 0.
- **How do you declare an array?** `int[] a = new int[5];` or `int a[] = {1,2,3};`
- **What is `a.length`?** A property (not a method) giving the array size.
- **Which sorting did you use?** Bubble sort: swaps adjacent elements; time complexity $O(n^2)$.
- **What does `Arrays.sort()` use?** Dual-Pivot Quicksort for primitives.
- **What is Scanner?** A class in `java.util` used to read input from the keyboard.
- **What is the enhanced for loop?** `for (int x : a)` iterates over each element without an index.
- **What is `ArrayIndexOutOfBoundsException`?** Thrown when accessing an index < 0 or $\geq \text{length}$.
- **What does `required` do?** Prevents form submission if the field is empty.
- **What is `pattern`?** A regular expression the input must match.
- **`minlength` vs `maxlength`?** Minimum and maximum number of characters allowed.
- **New HTML5 input types?** email, date, time, tel, number, url, range, color, file.
- **Why the same `name` for radio buttons?** So only one option in the group can be selected.
- **`submit` vs `reset`?** Submit sends form data; reset clears all fields.
- **What does CSS3 add?** Rounded corners, shadows, gradients, transitions, flexbox, grid, media queries.

Page 2 – Armstrong Numbers | Student Registration Form

Q1. Armstrong numbers between a range

An Armstrong number equals the sum of its digits, each raised to the power of the number of digits ($153 = 1^3 + 5^3 + 3^3$).

```
import java.util.*;
class Armstrong
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter start: ");
        int start = sc.nextInt();
        System.out.print("Enter end: ");
        int end = sc.nextInt();
        System.out.println("Armstrong numbers:");
        for (int n = start; n <= end; n++)
        {
            int digits = String.valueOf(n).length();
            int temp = n, sum = 0;
            while (temp > 0)
            {
                int d = temp % 10;
                sum = sum + (int) Math.pow(d, digits);
                temp = temp / 10;
            }
            if (sum == n)
                System.out.print(n + " ");
        }
    }
}
```

Sample: range 1-500 gives 1 2 3 4 5 6 7 8 9 153 370 371 407.

Q2. Student Registration Form (HTML5 + CSS3)

```
<!DOCTYPE html>
<html>
<head>
<title>Student Registration</title>
```

```

<style>
  .box
  {
    width: 650px;
    margin: auto;
    background: #ffe0e0;
    padding: 20px;
    font-family: Times;
  }
  h2
  {
    text-align: center;
  }
  td
  {
    padding: 6px;
    font-size: 14px;
  }
  .small
  {
    width: 60px;
  }
  .center
  {
    text-align: center;
  }
</style>
</head>
<body>
<div class="box">
  <h2>Student Registration Form</h2>
  <form>
    <table>
      <tr><td>Roll no. :</td><td><input type="number" required min="1">
</td></tr>
      <tr><td>Student name :</td>
        <td><input type="text" placeholder="First Name" required
pattern="[A-Za-z]+"> -
          <input type="text" placeholder="Last Name" required pattern="[A-Za-z]+"></td></tr>
      <tr><td>Father's name :</td><td><input type="text" required></td>
</tr>
      <tr><td>Date of birth :</td>

```

```

        <td><input class="small" type="number" placeholder="Day" min="1"
max="31" required> -
        <input class="small" type="number" placeholder="Month"
min="1" max="12" required> -
        <input class="small" type="number" placeholder="Year"
min="1990" max="2015" required>
        <i>(DD-MM-YYYY)</i></td></tr>
    <tr><td>Mobile no. :</td>
        <td><input class="small" type="text" value="+91" readonly> -
        <input type="tel" pattern="[0-9]{10}" required></td></tr>
    <tr><td>Email id :</td><td><input type="email" required></td></tr>
    <tr><td>Password :</td><td><input type="password" minlength="8"
required></td></tr>
    <tr><td>Gender :</td>
        <td><input type="radio" name="g" required> Male <input
type="radio" name="g"> Female</td></tr>
    <tr><td>Department :</td>
        <td><input type="checkbox"> CSE <input type="checkbox"> IT <input
type="checkbox"> ECE
        <input type="checkbox"> Civil <input type="checkbox">
Mech</td></tr>
    <tr><td>Course :</td>
        <td><select required>
            <option value="">--- Select Current Course's ---</option>
            <option>BSc CS</option><option>BCA</option>
<option>BTech</option>
        </select></td></tr>
    <tr><td>Student photo :</td><td><input type="file" accept="image/*">
</td></tr>
    <tr><td>City :</td><td><input type="text" required></td></tr>
    <tr><td>Address :</td><td><textarea rows="4" cols="35" required>
</textarea></td></tr>
</table>
<div class="center"><input type="submit" value="Register"></div>
</form>
</div>
</body>
</html>

```

Viva questions – Page 2

- **What is an Armstrong number?** A number equal to the sum of its digits each raised to the power of the digit count (e.g., 153, 370, 371, 407, 9474).
- **How do you extract digits?** `n % 10` gives the last digit; `n / 10` removes it.
- **What does `Math.pow()` return?** A double, so we cast it to int.
- **How do you count digits?** `String.valueOf(n).length()` or a loop dividing by 10.
- **while vs do-while?** while checks first; do-while runs at least once.
- **What does `placeholder` do?** Shows hint text inside an empty field.
- **What does `readonly` do?** Value shows but cannot be edited (still submitted, unlike `disabled`).
- **What are `min` and `max` ?** Limits for number/date inputs.
- **What does `accept="image/*"` do?** Restricts file chooser to image files.
- **Radio vs checkbox?** Radio allows one choice in a group; checkbox allows many.
- **What is client-side validation?** Checking input in the browser before sending to the server.

Page 5 – File Reverse & Case Change | Image Gallery

Q1. Display file contents in reverse order and change case

```
import java.io.*;
import java.util.*;
class ReverseFile
{
    public static void main(String[] args) throws IOException
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String fname = sc.nextLine();
        FileReader fr = new FileReader(fname);
        String data = "";
        int ch;
        while ((ch = fr.read()) != -1)
        {
            data = data + (char) ch;
        }
        fr.close();
    }
}
```

```

        System.out.println("Reversed with case changed:");
        for (int i = data.length() - 1; i >= 0; i--)
        {
            char c = data.charAt(i);
            if (Character.isUpperCase(c))
                c = Character.toLowerCase(c);
            else if (Character.isLowerCase(c))
                c = Character.toUpperCase(c);
            System.out.print(c);
        }
    }
}

```

If the file has "Hello Java", output is "AVAj OLLEh". Create a test file first (e.g., `test.txt`).

Q2. Image gallery with hover, responsive images and Grid

```

<!DOCTYPE html>
<html>
<head>
<title>Image Gallery</title>
<style>
    .gallery
    {
        display: grid;
        grid-template-columns: repeat(4, 1fr);
        gap: 10px;
    }
    .gallery img
    {
        width: 100%;
        height: 180px;
        object-fit: cover;
        transition: 0.3s;
    }
    .gallery img:hover
    {
        transform: scale(1.1);
    }
    @media (max-width: 600px)
    {
        .gallery
        {

```

```

        grid-template-columns: repeat(2, 1fr);
    }
}
</style>
</head>
<body>
    <h2>My Image Gallery</h2>
    <div class="gallery">
        
        
        
        
    </div>
</body>
</html>

```

Put any 4 images in the same folder (the @media rule makes it 2 columns on small screens) (or use <https://picsum.photos/300?1>, [?2](https://picsum.photos/300?2) and so on if internet is available).

Viva questions – Page 5

- **Byte stream vs character stream?** Byte streams (FileInputStream) read raw bytes; character streams (FileReader) read text characters.
- **Why does `read()` return -1?** It signals end of file.
- **Why `throws IOException`?** File operations can fail; IOException is a checked exception and must be handled or declared.
- **What is `FileNotFoundException`?** Thrown when the file name does not exist.
- **Why close the file?** To release system resources.
- **Other way to reverse?** `new StringBuilder(data).reverse()`.
- **What is `BufferedReader`?** Reads text efficiently using a buffer; has `readLine()`.
- **Grid vs Flexbox?** Grid is two-dimensional (rows + columns); Flexbox is one-dimensional.
- **What does `repeat(auto-fit, minmax(200px, 1fr))` do?** Fits as many 200px+ columns as the screen allows.
- **What is `transform: scale()`?** Enlarges or shrinks an element.
- **What is `transition`?** Makes the change smooth over time.
- **What is `object-fit: cover`?** Fills the box while keeping the aspect ratio, cropping extra.

- **What is the viewport meta tag?** Makes the page scale correctly on mobile devices.
- **Why the `alt` attribute?** Text shown if the image fails; used by screen readers.

Page 6 – Account → SavingAccount → AccountDetail | Hover Effects

Q1. Multilevel inheritance with constructors

```
class Account
{
    String custname;
    int accno;
    Account()
    {
    }
    Account(String n, int a)
    {
        custname = n;
        accno = a;
    }
}
class SavingAccount extends Account
{
    double savingbal, minbal;
    SavingAccount(String n, int a, double b, double m)
    {
        super(n, a);
        savingbal = b;
        minbal = m;
    }
}
class AccountDetail extends SavingAccount
{
    double depositamt, withdrawamt;
    AccountDetail(String n, int a, double b, double m, double d, double w)
    {
        super(n, a, b, m);
        depositamt = d;
        withdrawamt = w;
    }
    void display()
```

```

    {
        System.out.println("Customer Name: " + custname);
        System.out.println("Account No: " + accno);
        System.out.println("Saving Balance: " + savingbal);
        System.out.println("Minimum Balance: " + minbal);
        System.out.println("Deposit Amount: " + depositamt);
        System.out.println("Withdrawal Amount: " + withdrawalamt);
    }
    public static void main(String args[])
    {
        AccountDetail a = new AccountDetail("Sujata", 101, 10000, 1000, 5000,
2000);
        a.display();
    }
}

```

Save as `AccountDetail.java` (the class with main).

Q2. Button and link with :hover

```

<!DOCTYPE html>
<html>
<head>
<title>Hover Effect</title>
<style>
    button, a
    {
        padding: 10px;
        background: lightblue;
        color: black;
    }
    button:hover, a:hover
    {
        background: blue;
        color: white;
    }
</style>
</head>
<body>

```

```

    <button>Click Me</button>
    <a href="https://www.google.com">Visit Link</a>
</body>
</html>

```

Viva questions – Page 6

- **What is inheritance?** One class acquiring properties and methods of another using `extends`.
- **Which type is used here?** Multilevel inheritance ($A \rightarrow B \rightarrow C$).
- **Types of inheritance in Java?** Single, multilevel, hierarchical. Multiple inheritance only through interfaces.
- **Why no multiple inheritance with classes?** To avoid the diamond problem (ambiguity).
- **What is a constructor?** A special method with the class name and no return type, called when an object is created.
- **Default vs parameterized constructor?** Default takes no arguments; parameterized takes values to initialise fields.
- **What is `super()` ?** Calls the parent class constructor; must be the first statement.
- **Constructor overloading?** Multiple constructors with different parameters in one class.
- **What is a pseudo-class?** A keyword added to a selector for a special state, e.g. `:hover`, `:active`, `:visited`, `:focus`.
- **Link pseudo-class order?** LVHA: `:link`, `:visited`, `:hover`, `:active`.
- **What does `cursor: pointer` do?** Shows the hand icon on hover.

Page 7 – Driver Age Exception | `:first-child` & `:last-child`

Q1. Driver class with user-defined exception

```

import java.util.*;
class AgeException extends Exception
{
    AgeException(String msg)
    {
        super(msg);
    }
}

```

```

class Driver
{
    String license_no, name, address;
    int age;
    Driver(String l, String n, String a, int age)
    {
        license_no = l;
        name = n;
        address = a;
        this.age = age;
    }
    void display()
    {
        try
        {
            if (age < 18)
                throw new AgeException("Age is below 18 years");
            System.out.println("License No: " + license_no);
            System.out.println("Name: " + name);
            System.out.println("Address: " + address);
            System.out.println("Age: " + age);
        }
        catch (AgeException e)
        {
            System.out.println(e.getMessage());
        }
    }
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter age: ");
        Driver d = new Driver("L123", "Rahul", "Nashik", sc.nextInt());
        d.display();
    }
}

```

Only age is taken from the user to keep it short. Your notes' version was missing the closing brace of class Driver; it is fixed here.

Q2. List with :first-child and :last-child

```
<!DOCTYPE html>
<html>
<head>
<title>Programming Languages</title>
<style>
  li:first-child
  {
    color: red;
  }
  li:last-child
  {
    color: blue;
  }
</style>
</head>
<body>
  <h2>Programming Languages</h2>
  <ul>
    <li>Java</li>
    <li>Python</li>
    <li>C++</li>
    <li>JavaScript</li>
  </ul>
</body>
</html>
```

Viva questions – Page 7

- **What is an exception?** An abnormal event during execution that disrupts normal flow.
- **How do you create a user-defined exception?** Extend the `Exception` class and pass the message to `super(msg)`.
- **throw vs throws?** `throw` actually raises an exception object; `throws` declares that a method may raise it.
- **Checked vs unchecked?** Checked are verified at compile time (`IOException`); unchecked occur at runtime (`ArithmeticException`, `NullPointerException`).
- **Is your custom exception checked?** Yes, because it extends `Exception` (extending `RuntimeException` would make it unchecked).
- **What is `finally`?** A block that always executes, used for cleanup.

- **What does `getMessage()` return?** The message passed to the exception.
- **Can one try have many catch blocks?** Yes; the more specific exception must come first.
- **What does `:first-child` select?** An element that is the first child of its parent.
- **`:last-child`?** The last child of its parent.
- **`:nth-child(n)`?** Selects by position, e.g. `:nth-child(2)` or `:nth-child(even)`.
- **ul vs ol?** ul gives bullets (unordered); ol gives numbers (ordered).

Page 8 – Abstract Shape (printArea) | Image Swap with DOM

Q1. Abstract class Shape with method overriding

```
abstract class Shape
{
    int a, b;
    abstract void printArea();
}
class Rectangle extends Shape
{
    void printArea()
    {
        a = 10;
        b = 5;
        System.out.println("Rectangle Area = " + (a * b));
    }
}
class Triangle extends Shape
{
    void printArea()
    {
        a = 10;
        b = 5;
        System.out.println("Triangle Area = " + (a * b / 2));
    }
}
class Circle extends Shape
{
    void printArea()
    {
        a = 5;
```

```

        System.out.println("Circle Area = " + (3.14 * a * a));
    }
}
class Slip8_1
{
    public static void main(String args[])
    {
        Shape s;
        s = new Rectangle();
        s.printArea();
        s = new Triangle();
        s.printArea();
        s = new Circle();
        s.printArea();
    }
}

```

Output: 50, 25, 78.5. Save as Slip8_1.java.

Q2. Replace image on button click (DOM + event)

```

<!DOCTYPE html>
<html>
<head>
<title>Image Change</title>
</head>
<body>
    <h2>Change Image</h2>
    
    <br><br>
    <button onclick="changeImage()">Change Image</button>
    <script>
        function changeImage()
        {
            document.getElementById("img").src = "image2.jpg";
        }
    </script>
</body>
</html>

```

Keep `image1.jpg` and `image2.jpg` in the same folder. The `onclick` event calls `changeImage()`, which changes the image's `src` through the DOM.

Viva questions – Page 8

- **What is an abstract class?** A class declared with `abstract` that cannot be instantiated; it may have abstract and normal methods.
- **What is an abstract method?** A method with no body; subclasses must override it.
- **Can an abstract class have a constructor?** Yes, it is called through `super()` from the subclass.
- **What is method overriding?** A subclass gives its own version of a parent method with the same signature.
- **Overloading vs overriding?** Overloading: same name, different parameters, same class (compile time). Overriding: same signature in subclass (runtime).
- **What is runtime polymorphism / dynamic method dispatch?** A parent reference (`Shape s`) calls the child's method decided at runtime.
- **Abstract class vs interface?** Abstract class can have variables, constructors and concrete methods; interface supports multiple inheritance and has constants.
- **What is the DOM?** Document Object Model: the page as a tree of objects JavaScript can change.
- **What does `getElementById()` return?** The element with that id.
- **What is `addEventListener()` ?** Attaches a function to run when an event (click, mouseover) happens.
- **Other DOM selectors?** `querySelector()`, `getElementsByClassName()`, `getElementsByTagName()`.
- **What is an event?** An action like click, keypress, load or mouseover.

Page 9 – Employee Max Salary | Arrow Functions

Q1. Array of objects – employee with maximum salary

```
import java.util.*;
class Employee
{
    int id;
    String name;
    double salary;
    Employee(int i, String n, double s)
    {
        id = i;
```

```

        name = n;
        salary = s;
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many employees? ");
        int n = sc.nextInt();
        Employee[] e = new Employee[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Enter id, name, salary: ");
            e[i] = new Employee(sc.nextInt(), sc.next(), sc.nextDouble());
        }
        Employee max = e[0];
        for (int i = 1; i < n; i++)
        {
            if (e[i].salary > max.salary)
                max = e[i];
        }
        System.out.println("Highest salary: " + max.name + " (" + max.salary
+ ")");
    }
}

```

Input example: 1 Amit 25000 (name must be one word because `next()` reads a single word).

Q2. Arrow functions for arithmetic

```

// arrow.js -> run: node arrow.js
const add = (a, b) => a + b;
const sub = (a, b) => a - b;
const mul = (a, b) => a * b;
const div = (a, b) => b !== 0 ? a / b : "Cannot divide by zero";
let x = 20, y = 5;
console.log("Addition:", add(x, y));
console.log("Subtraction:", sub(x, y));
console.log("Multiplication:", mul(x, y));
console.log("Division:", div(x, y));

```

To run in a browser instead, wrap it in `<script>...</script>` inside an HTML file and check the console (F12).

Viva questions – Page 9

- **What is an array of objects?** An array whose elements are references to objects:
`Employee[] e = new Employee[n];`
- **Does `new Employee[n]` create objects?** No, it creates n null references; each object is created with `new Employee(...)`.
- **What is a class vs object?** Class is a blueprint; object is an instance of it.
- **`next()` vs `nextLine()`?** `next()` reads one word; `nextLine()` reads the whole line.
- **How do you find the max?** Assume the first is max, compare with each other element, update when larger.
- **What is an arrow function?** A shorter function syntax `(params) => expression` introduced in ES6.
- **Arrow vs normal function?** Arrow functions have no own `this` or `arguments` and cannot be used as constructors.
- **What is implicit return?** With a single expression and no braces, its value is returned automatically.
- **What is the ternary operator?** `condition ? valueIfTrue : valueIfFalse`.
- **What is ES6?** ECMAScript 2015, which added `let/const`, arrow functions, template literals, destructuring, classes, modules.
- **Dividing by zero in JS?** Gives `Infinity`, not an error, which is why we check.

Page 10 – abc.txt in Uppercase | Student Report with Template Literals

Q1. Read abc.txt and display in uppercase

```
import java.io.*;
class Slip10_1
{
    public static void main(String args[]) throws Exception
    {
        FileReader f = new FileReader("abc.txt");
        int ch;
        while ((ch = f.read()) != -1)
        {
            System.out.print(Character.toUpperCase((char) ch));
        }
        f.close();
    }
}
```

Create `abc.txt` in the same folder with some text before running.

Q2. Student report using template literals

```
<!DOCTYPE html>
<html>
<body>
<script>
    const name = "Rahul";
    const roll = 101;
    const marks = 85;
    let report = `
        Student Report<br>
        -----<br>
        Name : ${name}<br>
        Roll No : ${roll}<br>
        Marks : ${marks}
    `;
    document.write(report);
</script>
</body>
</html>
```

Viva questions – Page 10

- **What is `BufferedReader`?** A wrapper that reads text efficiently using a buffer; `readLine()` reads one line.
- **What does `readLine()` return at end of file?** `null`.
- **Why wrap `FileReader` inside `BufferedReader`?** `FileReader` reads one character at a time; buffering is faster and gives `readLine()`.
- **What does `toUpperCase()` do?** Returns a new string in capital letters (strings are immutable).
- **What is the `java.io` package?** Classes for input and output: `File`, `FileReader`, `FileWriter`, `BufferedReader`, streams.
- **Order of catch blocks?** `FileNotFoundException` before `IOException`, since it is a subclass.
- **What is try-with-resources?** `try (BufferedReader br = ...) closes the resource automatically.`
- **What are template literals?** Strings in backticks (```) that allow `${expression}` and multi-line text.
- **What is string interpolation?** Inserting variable values inside a string using `${}`.

- **Template literal vs normal string?** No need for `+` concatenation, and line breaks are kept as written.
- **What does `toFixed(2)` do?** Rounds a number to 2 decimal places and returns a string.

Page 11 – Vehicle (LMV/HMV) | Total & Average with Arrow Function

Q1. Vehicle superclass with two subclasses

```
import java.util.*;
class Vehicle
{
    String company;
    double price;
    Vehicle(String c, double p)
    {
        company = c;
        price = p;
    }
    void display()
    {
        System.out.print("Company: " + company + ", Price: " + price);
    }
}
class LightMotorVehicle extends Vehicle
{
    double mileage;
    LightMotorVehicle(String c, double p, double m)
    {
        super(c, p);
        mileage = m;
    }
    void display()
    {
        System.out.print("[LMV] ");
        super.display();
        System.out.println(", Mileage: " + mileage + " km/l");
    }
}
class HeavyMotorVehicle extends Vehicle
{
```

```

    double capacity;
    HeavyMotorVehicle(String c, double p, double cap)
    {
        super(c, p);
        capacity = cap;
    }
    void display()
    {
        System.out.print("[HMOV] ");
        super.display();
        System.out.println(", Capacity: " + capacity + " tons");
    }
}
class VehicleDemo
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("How many vehicles? ");
        int n = sc.nextInt();
        Vehicle[] v = new Vehicle[n];
        for (int i = 0; i < n; i++)
        {
            System.out.print("Type (1 = Light, 2 = Heavy): ");
            int type = sc.nextInt();
            System.out.print("Company and price: ");
            String c = sc.next();
            double p = sc.nextDouble();
            if (type == 1)
            {
                System.out.print("Mileage: ");
                v[i] = new LightMotorVehicle(c, p, sc.nextDouble());
            }
            else
            {
                System.out.print("Capacity in tons: ");
                v[i] = new HeavyMotorVehicle(c, p, sc.nextDouble());
            }
        }
        System.out.println("\n--- Vehicle Details ---");
        for (Vehicle x : v)
        {

```

```

        x.display();
    }
}

```

Q2. Total and average marks with arrow function

```

<!DOCTYPE html>
<html>
<body>
<script>
    const total = (a, b, c) => a + b + c;
    const average = (a, b, c) => (a + b + c) / 3;
    const m1 = 80, m2 = 70, m3 = 90;
    document.write(`
        Total Marks = ${total(m1, m2, m3)}<br>
        Average Marks = ${average(m1, m2, m3)}
    `);
</script>
</body>
</html>

```

Viva questions – Page 11

- **What type of inheritance is this?** Hierarchical: one parent (Vehicle) with two children.
- **Why is `Vehicle[]` able to hold both types?** A parent reference can refer to any child object (upcasting).
- **Which `display()` runs?** The child's overridden version, chosen at runtime (dynamic method dispatch).
- **What does `super.display()` do?** Calls the parent class version of the method.
- **What is `super`?** A keyword referring to the immediate parent class (its constructor, methods or fields).
- **What is upcasting?** Assigning a child object to a parent reference; done automatically.
- **What is `instanceof`?** Checks whether an object belongs to a class, e.g. `v instanceof LightMotorVehicle`.
- **Why no `return` in the arrow functions?** A single expression without braces is returned automatically (implicit return).
- **What is `${}` inside backticks?** String interpolation in a template literal.

- **What does `document.write()` do?** Writes output directly into the web page.

Page 12 – Package game (Indoor/Outdoor) | Product Bill

Q1. Package game with Indoor and Outdoor classes

Folder structure: a folder named `game` containing `Indoor.java` and `Outdoor.java`, and `GameDemo.java` outside it.

```
// File: game/Indoor.java
package game;
public class Indoor
{
    String game, players;
    public Indoor()
    {
        game = "Chess";
        players = "Anand, Magnus";
    }
    public Indoor(String g, String p)
    {
        game = g;
        players = p;
    }
    public void display()
    {
        System.out.println("Indoor Game: " + game + " | Players: " +
players);
    }
}
```

```
// File: game/Outdoor.java
package game;
public class Outdoor
{
    String game, players;
    public Outdoor()
    {
        game = "Cricket";
        players = "Virat, Rohit, Bumrah";
    }
}
```

```

    public Outdoor(String g, String p)
    {
        game = g;
        players = p;
    }
    public void display()
    {
        System.out.println("Outdoor Game: " + game + " | Players: " +
players);
    }
}

```

```

// File: GameDemo.java (outside the game folder)
import game.*;
class GameDemo
{
    public static void main(String args[])
    {
        new Indoor().display();
        new Indoor("Carrom", "Amit, Sumit").display();
        new Outdoor().display();
        new Outdoor("Football", "Sunil, Messi").display();
    }
}

```

The constructors must be `public`. In your notes they had no modifier, so `new Indoor("Carrom")` from outside the package would give a compile error.

Compile and run: `javac game/Indoor.java game/Outdoor.java GameDemo.java` then `java GameDemo`.

Q2. Product bill with arrow function and template literals

```
<!DOCTYPE html>
<html>
<body>
<script>
    const total = (price, qty) => price * qty;
    const product = "Pen";
    const price = 20;
    const qty = 5;
    document.write(`
        Product : ${product}<br>
        Price : ${price}<br>
        Quantity : ${qty}<br>
        Total : ${total(price, qty)}
    `);
</script>
</body>
</html>
```

Output: Total : 100.

Viva questions – Page 12

- **What is a package?** A group of related classes and interfaces, like a folder; avoids name conflicts.
- **Built-in vs user-defined packages?** Built-in: java.lang, java.util, java.io. User-defined: created with `package name; .`
- **Which package is imported by default?** java.lang.
- **Why must the classes be `public`?** So they can be accessed from outside the package.
- **Access modifiers?** private (class only), default (package only), protected (package + subclasses), public (everywhere).
- **What does `import game.*; do?`** Imports all public classes of the game package.
- **What does `javac -d . do?`** Creates the package folder automatically while compiling.
- **Must `package` be the first line?** Yes, the first statement (comments allowed above).
- **What is string interpolation?** Embedding `${expression}` inside a template literal.
- **Can template literals hold expressions?** Yes, e.g. `${price * qty}`.

Page 13 – Clock Class (AM/PM) | Swap with Array Destructuring

Q1. Clock class: accept, validate, AM/PM

```
import java.util.*;
class Clock
{
    int hh, mm, ss;
    Clock()
    {
        hh = 0;
        mm = 0;
        ss = 0;
    }
    Clock(int h, int m, int s)
    {
        hh = h;
        mm = m;
        ss = s;
    }
    boolean isValid()
    {
        return hh >= 0 && hh <= 23 && mm >= 0 && mm <= 59 && ss >= 0 && ss <=
59;
    }
    void showAmPm()
    {
        String period = (hh < 12) ? "AM" : "PM";
        int h = hh % 12;
        if (h == 0)
            h = 12;
        System.out.printf("Time: %02d:%02d:%02d %s\n", h, mm, ss, period);
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter hours minutes seconds (24-hr): ");
        Clock c = new Clock(sc.nextInt(), sc.nextInt(), sc.nextInt());
        if (c.isValid())
            c.showAmPm();
    }
}
```

```

        else
            System.out.println("Invalid time!");
    }
}

```

Example: input 15 30 45 prints 03:30:45 PM; input 25 10 10 prints Invalid time.

Q2. Swap two numbers using array destructuring

```

// swap.js -> run: node swap.js
let a = 10, b = 20;
console.log(`Before swap: a = ${a}, b = ${b}`);
[a, b] = [b, a];
console.log(`After swap: a = ${a}, b = ${b}`);

```

Viva questions – Page 13

- **How do you convert 24-hour to 12-hour?** `h % 12`, and if the result is 0 use 12; below 12 is AM, otherwise PM.
- **Why validate input?** Hours must be 0–23, minutes and seconds 0–59.
- **What does `%02d` mean?** Print an integer in at least 2 digits, padded with 0.
- **printf vs println?** printf prints formatted output; println prints and moves to a new line.
- **What is encapsulation?** Keeping data and methods together in a class; making fields private with getters/setters.
- **What is a boolean method?** Returns true or false, e.g. `isValid()`.
- **What is destructuring?** Unpacking values from arrays or objects into separate variables.
- **How does `[a, b] = [b, a]` swap?** The right side builds a temporary array [20, 10], which is then unpacked into a and b.
- **Other ways to swap without a third variable?** `a = a + b; b = a - b; a = a - b;` or using XOR.
- **Can you skip values in destructuring?** Yes: `let [x, , z] = [1, 2, 3];`
- **Default values in destructuring?** `let [p = 5, q = 7] = [1];` gives p = 1, q = 7.

Page 19 – Count Characters, Words, Lines | async/await Login

Q1. Count characters, words and lines in a file

```
import java.io.*;
import java.util.*;
class FileCount
{
    public static void main(String[] args) throws IOException
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter file name: ");
        String fname = sc.nextLine();
        BufferedReader br = new BufferedReader(new FileReader(fname));
        int chars = 0, words = 0, lines = 0;
        String line;
        while ((line = br.readLine()) != null)
        {
            lines++;
            chars = chars + line.length();
            if (!line.trim().isEmpty())
                words = words + line.trim().split("\\s+").length;
        }
        br.close();
        System.out.println("Characters: " + chars);
        System.out.println("Words: " + words);
        System.out.println("Lines: " + lines);
    }
}
```

Q2. Simulated login using async/await with try-catch

```
// login.js -> run: node login.js
async function login(user, pass)
{
    if (user == "admin" && pass == "1234")
        return "Login Successful";
    else
        throw "Invalid Username or Password";
}
async function checkLogin(user, pass)
{

```

```

    try
    {
        let result = await login(user, pass);
        console.log(result);
    }
    catch (error)
    {
        console.log("Error: " + error);
    }
}
checkLogin("admin", "1234");    // success
checkLogin("guest", "0000");    // error

```

The Java Q1 above keeps the original version: your notes' `split(" ")` miscounts words when a line is empty or has double spaces.

Viva questions – Page 19

- **What does `split("\\s+")` do?** Splits text on one or more spaces/tabs to get words.
- **Why `trim()` ?** Removes leading and trailing spaces so extra spaces don't count as words.
- **Does `line.length()` count newline characters?** No, `readLine()` strips them.
- **What is a regular expression?** A pattern to match text; `\s` = whitespace, `+` = one or more.
- **What is a Promise?** An object for a value available later: pending, fulfilled (resolved) or rejected.
- **What is `async` ?** Marks a function that always returns a Promise and allows `await` inside.
- **What does `return` inside an `async` function do?** Resolves its Promise with that value.
- **What does `throw` inside an `async` function do?** Rejects its Promise, so control goes to `catch`.
- **What is `await` ?** Pauses the `async` function until the Promise settles, then gives its result.
- **Synchronous vs asynchronous?** Synchronous runs line by line and blocks; asynchronous lets other code run while waiting.
- **`==` vs `===` ?** `==` converts types before comparing; `===` checks value and type (strict).

Page 21 – College → Department | Node.js path Module

Q1. College parent class, Department derived class

```
class College
{
    int cno;
    String cname, caddr;
    College(int no, String name, String addr)
    {
        cno = no;
        cname = name;
        caddr = addr;
    }
}
class Department extends College
{
    int dno;
    String dname;
    Department(int cno, String cname, String caddr, int dno, String dname)
    {
        super(cno, cname, caddr);
        this.dno = dno;
        this.dname = dname;
    }
    void display()
    {
        System.out.println("College No: " + cno);
        System.out.println("College Name: " + cname);
        System.out.println("College Address: " + caddr);
        System.out.println("Department No: " + dno);
        System.out.println("Department Name: " + dname);
    }
    public static void main(String args[])
    {
        Department d = new Department(101, "Ashoka College", "Nashik", 1,
"Computer Science");
        d.display();
    }
}
```

Q2. Node.js path module operations

```
// Slip21_2.js -> run: node Slip21_2.js
const path = require("path");
let p = "C:/College/Java/program.js";
console.log("Join:", path.join("C:/College", "Java", "program.js"));
console.log("Resolve:", path.resolve("program.js"));
console.log("Directory:", path.dirname(p));
console.log("File Name:", path.basename(p));
console.log("Extension:", path.extname(p));
```

Viva questions – Page 21

- **What is single inheritance?** One child class extends one parent class.
- **Use of `this` here?** Distinguishes instance variables from parameters with the same name.
- **Why only one `display()` in Department?** Department inherits `cno`, `cname` and `caddr`, so it can print College details too.
- **Is the constructor inherited?** No; the child calls it using `super(...)`.
- **What is the path module?** A core Node.js module for working with file and directory paths.
- **`path.join()` vs `path.resolve()`?** `join` simply combines parts; `resolve` gives an absolute path from the current directory.
- **What do `dirname`, `basename`, `extname` return?** Folder path, file name, file extension.
- **What does `path.parse()` return?** An object with `root`, `dir`, `base`, `ext` and `name`.
- **What is `require()`?** Loads a module in CommonJS style.
- **What are `__dirname` and `__filename`?** Absolute path of the current folder and current file.
- **Core modules in Node?** `fs`, `path`, `http`, `os`, `events`, `url`.

Page 22 – Product with Interface & Object Count | Async fs Operations

Q1. Interface + Product class with object count

```
interface Display
{
    void display();
}
class Product implements Display
{
    int product_id, product_quantity;
    String product_name;
    double product_cost;
    static int count = 0;
    Product()
    {
        count++;
    }
    Product(int id, String name, double cost, int qty)
    {
        product_id = id;
        product_name = name;
        product_cost = cost;
        product_quantity = qty;
        count++;
    }
    public void display()
    {
        System.out.println("ID: " + product_id + ", Name: " + product_name
            + ", Cost: " + product_cost + ", Qty: " + product_quantity);
    }
    public static void main(String args[])
    {
        Product p1 = new Product();
        Product p2 = new Product(101, "Pen", 20, 5);
        Product p3 = new Product(102, "Book", 100, 3);
        p1.display();
        p2.display();
    }
}
```

```

        p3.display();
        System.out.println("Object Count = " + count);
    }
}

```

Q2. Asynchronous file operations with fs

```

// Slip22_2.js -> run: node Slip22_2.js
const fs = require("fs");
fs.writeFile("test.txt", "Hello Node.js", (err) =>
{
    if (err) throw err;
    console.log("File written");
    fs.readFile("test.txt", "utf8", (err, data) =>
    {
        if (err) throw err;
        console.log("File content:", data);
        fs.appendFile("test.txt", "\nWelcome!", (err) =>
        {
            if (err) throw err;
            console.log("File updated");
        });
    });
});
});

```

Viva questions – Page 22

- **What is an interface?** A contract of abstract methods that a class must implement using `implements`.
- **Why must `display()` be public?** Interface methods are public by default, and you cannot reduce visibility.
- **Can a class implement many interfaces?** Yes, this is how Java supports multiple inheritance.
- **Variables in an interface?** Always `public static final` (constants).
- **What is a static variable?** One copy shared by all objects of the class.
- **How does object count work?** `count++` in every constructor; since count is static it keeps the total.
- **Why access as `Product.count`?** Static members belong to the class, not to an object.

- **Default methods in interfaces?** Since Java 8, interfaces can have `default` and `static` methods with bodies.
- **What is the fs module?** Node's core module for reading, writing, deleting and managing files.
- **What is a callback?** A function passed to another function to run when the task finishes.
- **Error-first callback?** The first parameter is `err`; if not null, the operation failed.
- **What is callback hell?** Deeply nested callbacks; solved with Promises (`fs.promises`) or `async/await`.
- **What does `unlink` do?** Deletes a file.

Page 23 – Continent → Country → State | fs Create, Write, Read, Append

Q1. Multilevel inheritance

```
class Continent
{
    String continent;
    Continent(String c)
    {
        continent = c;
    }
}
class Country extends Continent
{
    String country;
    Country(String c, String co)
    {
        super(c);
        country = co;
    }
}
class State extends Country
{
    String state, place;
    State(String c, String co, String s, String p)
    {
        super(c, co);
    }
}
```

```

        state = s;
        place = p;
    }
    void display()
    {
        System.out.println("Place      : " + place);
        System.out.println("State      : " + state);
        System.out.println("Country   : " + country);
        System.out.println("Continent : " + continent);
    }
    public static void main(String[] args)
    {
        State s = new State("Asia", "India", "Maharashtra", "Nashik");
        s.display();
    }
}

```

Q2. Create, write, read and append a file with fs

```

// filedemo.js -> run: node filedemo.js
const fs = require("fs");
// 1. Create + write
fs.writeFileSync("student.txt", "Name: Rahul\n");
console.log("File created and written");
// 2. Read
let data = fs.readFileSync("student.txt", "utf8");
console.log("After write:\n" + data);
// 3. Append
fs.appendFileSync("student.txt", "Course: BSc CS\n");
console.log("Data appended");
// 4. Read again
data = fs.readFileSync("student.txt", "utf8");
console.log("After append:\n" + data);

```

If the examiner asks for the async version, use `fs.writeFile`, `fs.readFile`, `fs.appendFile` with callbacks (see Page 22).

Viva questions – Page 23

- **What is multilevel inheritance?** A chain: State extends Country, which extends Continent.
- **Constructor call order?** Parent first: Continent → Country → State.

- **How does State access `continent`?** It is inherited through Country from Continent.
- **Multilevel vs hierarchical?** Multilevel is a chain; hierarchical is one parent with many children.
- **What is the root of all Java classes?** `java.lang.Object`.
- **What if `super(...)` isn't written?** Java inserts `super()` automatically; it fails if the parent has no default constructor.
- **`writeFile` vs `appendFile`?** `writeFile` overwrites (or creates) the file; `appendFile` adds at the end.
- **What is `"utf8"` in `readFile`?** Encoding; without it you get a Buffer instead of a string.
- **Does `writeFile` create a file?** Yes, if it does not exist.
- **What is a Buffer?** Raw binary data in Node.js.
- **Sync vs async methods?** Sync (`...Sync`) block until done; async use callbacks and don't block.

Page 24 – Abstract Shape → Cylinder | Sync vs Async File Operations

Q1. Abstract Shape with area and volume, Cylinder subclass

```
abstract class Shape
{
    abstract void area();
    abstract void volume();
}
class Cylinder extends Shape
{
    double r, h;
    Cylinder(double r, double h)
    {
        this.r = r;
        this.h = h;
    }
    void area()
    {
        System.out.println("Area = " + 2 * Math.PI * r * (r + h));
    }
    void volume()
    {
```

```

        System.out.println("Volume = " + Math.PI * r * r * h);
    }
    public static void main(String args[])
    {
        Shape s = new Cylinder(5, 10);
        s.area();
        s.volume();
    }
}

```

Formulas: surface area = $2\pi r(r + h)$; volume = $\pi r^2 h$. For $r = 5$, $h = 10$: area ≈ 471.24 , volume ≈ 785.40 . (Your notes had a stray backtick after ``class Cylinder extends Shape {``, which is removed here.)

Q2. Synchronous vs asynchronous file operations

```

// Slip24_2.js -> run: node Slip24_2.js
const fs = require("fs");
fs.writeFileSync("data.txt", "Hello Node.js"); // sample file
console.log("Start");
console.log("Synchronous:");
let data1 = fs.readFileSync("data.txt", "utf8");
console.log(data1);
console.log("Asynchronous:");
fs.readFile("data.txt", "utf8", (err, data2) =>
{
    if (err) throw err;
    console.log(data2);
});
console.log("End");

```

Output: Start → Synchronous: → Hello Node.js → Asynchronous: → End → Hello Node.js.
The sync read prints its content before moving on (blocking). The async read prints after End (non-blocking).

Viva questions – Page 24

- **Why is Shape abstract?** Area and volume depend on the actual shape, so subclasses must define them.
- **Can you create `new Shape()`?** No, abstract classes cannot be instantiated.
- **What is `Math.PI`?** A constant for π (3.14159...) in `java.lang.Math`.

- **What is abstraction?** Showing only essential features and hiding implementation details.
- **Must a subclass override all abstract methods?** Yes, otherwise it must be declared abstract too.
- **What is blocking code?** Code that stops further execution until it finishes (readFileSync).
- **Why is async preferred in servers?** It doesn't block the single thread, so many requests can be handled.
- **What is the event loop?** Node's mechanism that runs callbacks when async tasks finish.
- **When is sync acceptable?** At program startup or in small scripts, e.g. reading a config file.
- **What is libuv?** The C library Node uses for the event loop and thread pool behind async I/O.

Page 25 – Calculator Interface | Directory Management + JSON

Q1. Interface Calculator implemented by SimpleCalc

```
interface Calculator
{
    int add(int a, int b);
    int subtract(int a, int b);
}
class SimpleCalc implements Calculator
{
    public int add(int a, int b)
    {
        return a + b;
    }
    public int subtract(int a, int b)
    {
        return a - b;
    }
    public static void main(String[] args)
    {
        Calculator c = new SimpleCalc();
```

```

        System.out.println("Addition      = " + c.add(20, 10));
        System.out.println("Subtraction = " + c.subtract(20, 10));
    }
}

```

Q2. Directory operations and JSON with fs

```

// Slip25_2.js -> run: node Slip25_2.js
const fs = require("fs");
fs.mkdirSync("MyFolder");
console.log("Directory created");
console.log("Contents:", fs.readdirSync("."));
fs.renameSync("MyFolder", "NewFolder");
console.log("Directory renamed");
let student = { name: "Rahul", age: 20, course: "BSc CS" };
fs.writeFileSync("student.json", JSON.stringify(student, null, 2));
console.log("JSON file written");
let obj = JSON.parse(fs.readFileSync("student.json", "utf8"));
console.log("Student:", obj);
fs.rmdirSync("NewFolder");
console.log("Directory deleted");

```

Run it once in a clean folder: `mkdirSync` throws an error if `MyFolder` already exists. The Java Q1 above keeps the original version, because your notes' `subtract()` was missing its opening brace.

Viva questions – Page 25

- **Interface vs abstract class?** Interface methods are abstract by default and a class can implement many; abstract class allows fields, constructors and only single inheritance.
- **Why `Calculator c = new SimpleCalc();` ?** An interface reference can hold any implementing object (loose coupling, polymorphism).
- **Can we create an object of an interface?** No.
- **What happens if a method isn't implemented?** Compile error, unless the class is abstract.
- **What is a functional interface?** An interface with exactly one abstract method; usable with lambdas.
- **`mkdirSync`, `readdirSync`, `renameSync`, `rmdirSync`?** Create a directory, list contents, rename, delete.

- **Why { recursive: true } ?** To delete a non-empty folder (or create nested folders with mkdir).
- **What is fs.existsSync() ?** Returns true if the path exists.
- **What is JSON?** JavaScript Object Notation: a text format for structured data using key-value pairs.
- **JSON.stringify vs JSON.parse?** stringify converts an object to a JSON string; parse converts a JSON string to an object.
- **What do null, 2 mean in stringify?** No replacer, indent with 2 spaces for readable output.

Page 26 – Student Attendance Exception | fs Copy, Rename, Delete

Q1. Student eligibility with user-defined exception

```
class AttendanceException extends Exception
{
    AttendanceException(String msg)
    {
        super(msg);
    }
}
class Student
{
    String student_name;
    int student_rollno, total_lectures, attended_lectures;
    Student(String name, int roll, int total, int attended)
    {
        student_name = name;
        student_rollno = roll;
        total_lectures = total;
        attended_lectures = attended;
    }
    void display()
    {
        try
        {
            double per = (attended_lectures * 100.0) / total_lectures;
            if (per < 75)
                throw new AttendanceException("Student is Not Eligible for
Exam");
```

```

        System.out.println("Name: " + student_name);
        System.out.println("Roll No: " + student_rollno);
        System.out.println("Attendance: " + per + "%");
    }
    catch (AttendanceException e)
    {
        System.out.println(e.getMessage());
    }
}
public static void main(String args[])
{
    Student s = new Student("Rahul", 101, 100, 80);    // try 100, 60 for
exception
    s.display();
}
}

```

Note **100.0** (not 100) so the percentage is calculated as a decimal, avoiding integer division.

Q2. Copy, rename and delete files asynchronously

```

// fileops.js -> run: node fileops.js
const fs = require("fs");
fs.writeFileSync("original.txt", "Sample data");    // setup file
fs.copyFile("original.txt", "copy.txt", (err) =>
{
    if (err) return console.log("Copy failed:", err.message);
    console.log("File copied successfully");
    fs.rename("copy.txt", "renamed.txt", (err) =>
    {
        if (err) return console.log("Rename failed:", err.message);
        console.log("File renamed successfully");
        fs.unlink("renamed.txt", (err) =>
        {
            if (err) return console.log("Delete failed:", err.message);
            console.log("File deleted successfully");
        });
    });
});
// Error demo: file that does not exist

```

```
fs.unlink("nofile.txt", (err) =>
{
    if (err) console.log("Error:", err.message);
});
```

Viva questions – Page 26

- **How is attendance calculated?** $\text{attended} \times 100 / \text{total}$.
- **Why 100.0 ?** int / int gives an integer result; using a double keeps decimals.
- **Where is the exception thrown and caught?** Both inside `display()`: thrown with `throw`, caught in its try-catch.
- **What happens if not caught?** The program terminates and prints a stack trace.
- **What is exception propagation?** An uncaught exception passes up the call stack to the caller.
- **What is `printStackTrace()` ?** Prints the exception and the line numbers where it occurred.
- **Which fs methods did you use?** `copyFile` (copy), `rename` (rename/move), `unlink` (delete).
- **Why are they nested?** Each step must finish before the next one starts.
- **What is `err.message` ?** A readable error text, e.g. ENOENT: no such file or directory.
- **What does ENOENT mean?** Error NO ENTry: the file or folder does not exist.
- **Promise-based alternative?** `fs.promises.copyFile()` with `async/await`.

Page 29 – Zero Exception + Prime (static) | fs.stat()

Q1. User-defined exception for 0, otherwise check prime

```
import java.util.*;
class ZeroException extends Exception
{
    ZeroException(String msg)
    {
        super(msg);
    }
}
class PrimeCheck
{
```

```
static boolean isPrime(int n)
{
    if (n < 2)
        return false;
    for (int i = 2; i <= n / 2; i++)
    {
        if (n % i == 0)
            return false;
    }
    return true;
}

public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    System.out.print("Enter a number: ");
    int n = sc.nextInt();
    try
    {
        if (n == 0)
            throw new ZeroException("Number is 0");
        if (isPrime(n))
            System.out.println(n + " is Prime");
        else
            System.out.println(n + " is Not Prime");
    }
    catch (ZeroException e)
    {
        System.out.println("Exception: " + e.getMessage());
    }
}
```

Q2. File statistics using fs.stat()

```
// Slip29_2.js -> run: node Slip29_2.js
const fs = require("fs");
fs.writeFileSync("data.txt", "Hello stats"); // sample file
fs.stat("data.txt", (err, stats) =>
{
    if (err) return console.log("Error:", err.message);
    console.log("File Size:", stats.size, "bytes");
    console.log("Created:", stats.birthtime);
    console.log("Accessed:", stats.atime);
    console.log("Modified:", stats.mtime);
    if (stats.isFile())
        console.log("Type: File");
    else if (stats.isDirectory())
        console.log("Type: Directory");
});
```

The Java Q1 above keeps the original version: it is shorter and checks divisors only up to $n/2$.

Viva questions – Page 29

- **What is a prime number?** A number greater than 1 divisible only by 1 and itself.
- **Why loop till $n/2$ (or $\sqrt{n}$)?** No divisor (other than n) is larger than $n/2$; checking up to $\sqrt{n}$ is even faster.
- **What does `static` mean?** The member belongs to the class, not objects; called without creating an object.
- **Why is `isPrime` static?** So main (also static) can call it directly.
- **Can a static method use instance variables?** No, not directly; only static members.
- **What is a static block?** `static { }` runs once when the class loads.
- **Why a custom exception for 0?** The language has no built-in rule that 0 is an error here; we define our own business rule.
- **What is `fs.stat()` ?** Gets information about a file or directory as a Stats object.
- **`atime`, `mtime`, `ctime`, `birthtime`?** Last access, last content modification, last status change, creation time.
- **`isFile()` vs `isDirectory()`?** Tell whether the path is a regular file or a folder.
- **Sync version?** `fs.statSync(path)` .